

RAG-NIMBF: A Non-Invasive, Modular Benchmarking Framework for Automated RAG Configuration Analysis

Niclas Hart¹ and Nicolas Weeger¹

Ansbach University of Applied Sciences

Abstract. Retrieval-Augmented Generation (RAG) [14] has become the dominant paradigm for grounding large language model outputs in external knowledge, yet practitioners face a vast configuration space—chunking, embedding, retrieval, reranking, prompting, generation model—with limited guidance on how components interact. We present RAG-NIMBF, a modular benchmarking framework that attaches to existing RAG pipelines as a non-invasive evaluation layer, exploring the configuration space via cartesian-product sweeps with MLflow tracking end-to-end. Evaluation combines LLM-based quality (RAGAS), IR measures (nDCG, Hit@k, Recall@k), NLG metrics (BERTScore, METEOR, ROUGE-L, BLEU), and runtime indicators (latency, throughput). We validate the framework on 182 configurations across seven models on SQuAD, revealing non-obvious interactions: MMR retrieval improves nDCG@5 by 8–9pp and Recall@5 by up to 31pp over similarity search; detailed prompt templates raise faithfulness by up to 12pp for capable models; and larger generation models trade latency for answer quality. The framework enables systematic, comparable evaluation of RAG components with minimal integration effort.

Keywords: Retrieval-Augmented Generation · Benchmarking Framework · RAG Evaluation · Configuration Analysis

1 Introduction

Retrieval-Augmented Generation (RAG) [14] has emerged as the standard architecture for building knowledge-grounded question-answering systems. Rather than relying solely on parametric knowledge stored in model weights, RAG pipelines retrieve relevant documents from an external corpus and condition answer generation on the retrieved context. This approach reduces hallucination [28] and enables domain-specific applications without costly fine-tuning.

A production RAG pipeline involves multiple interacting components: a *chunking strategy* splits source documents into retrievable segments; an *embedding model* maps chunks and queries to a shared vector space; a *retrieval method* selects the most relevant chunks; an optional *reranker* re-orders them; a *prompt template* frames the generation task; and a *generation model* produces the final

answer. Each component offers multiple design choices, and the resulting configuration space is combinatorially large. Small changes in chunk size, retrieval strategy, or prompt wording can shift faithfulness scores by more than ten percentage points [4], yet practitioners often select configurations based on intuition rather than systematic evaluation.

Several evaluation frameworks have been proposed. RAGAS [7] provides automated metrics for answer quality and context usage. ARES [27] offers prediction-powered evaluation. RAGChecker [26] diagnoses fine-grained retrieval and generation failures. However, these tools evaluate a *fixed* pipeline configuration. They do not systematically sweep the configuration space, compare component interactions, or track experiments for reproducibility. Practitioners are left to orchestrate multi-configuration benchmarks manually, often with ad-hoc scripts that lack structured metric collection.

We address this gap with RAG-NIMBF (Non-Invasive Modular Benchmarking Framework), a framework designed around three principles:

1. **Non-invasive integration.** The framework wraps existing RAG components without restructuring the pipeline. It connects to Ollama ¹ and OpenAI-compatible endpoints (vLLM, TGI, LM Studio) through provider routing, requiring only model identifier strings.
2. **Systematic configuration exploration.** A declarative `.env` configuration specifies parameter ranges for all components. The framework generates the cartesian product and benchmarks every combination, producing structured results suitable for direct comparison.
3. **Hybrid evaluation.** Each run is assessed through LLM-based quality metrics (faithfulness, context recall, semantic similarity via RAGAS), established IR measures (nDCG@k, Hit@k, Recall@k), NLG metrics (BERTScore, METEOR, ROUGE-L, BLEU), and runtime indicators (time-to-first-token, tokens per second).

We validate RAG-NIMBF by benchmarking 182 configurations across seven generation models spanning local, cloud-hosted, and API deployments on 100-question samples from the SQuAD dataset [24]. All experiments are tracked via MLflow for full reproducibility.

The main contributions are:

- A modular, non-invasive benchmarking framework for systematic RAG configuration analysis.
- A hybrid evaluation methodology combining LLM-based, IR, and NLG metrics with runtime measurements.
- An empirical study of 182 configurations revealing non-obvious interactions between chunking, retrieval, prompting, and generation components.

The remainder of this paper covers related work, framework architecture, experimental setup, results, validity threats, and conclusions.

¹ <https://ollama.com>

2 Related Work

RAG was introduced by Lewis et al. [14], combining dense passage retrieval [11] with sequence-to-sequence generation. Subsequent systems extend retrieval into pre-training, self-reflection, dynamic retrieval, graph-based reasoning, and evidence checking [9, 1, 10, 6, 15]. Surveys [8, 33] show that chunking, retrieval, and prompt design affect output quality, but offer no tooling for systematic configuration comparison.

Several frameworks evaluate a *single* pipeline configuration. RAGAS [7] introduced LLM-based metrics (faithfulness, answer relevancy, context precision/recall). ARES [27] uses prediction-powered inference without gold references. RAGChecker [26] decomposes performance into retrieval/generation diagnostics. Chen et al. [4] benchmark LLMs on RAG but evaluate only model-level differences. Beyond metric suites, observability platforms (LangSmith [12], Langfuse [13]) provide tracing and cost telemetry without cartesian-product sweeps or IR/NLG metrics; TruLens [30] offers the RAG Triad for single-config evaluation; DeepEval [5] adapts pytest-style workflows for LLM regression tests. None systematically sweep the multi-component configuration space.

Individual components have been studied in isolation: lost-in-the-middle effects [17] motivate chunking/retrieval strategies; MMR [3] balances relevance and diversity; MTEB [18] benchmarks embedding models; Sentence-BERT [25] enables efficient semantic similarity. These studies examine one component at a time, leaving interaction effects across the full pipeline unexplored.

In the broader NLP ecosystem, benchmarks like GLUE, SuperGLUE, and MTEB [18] standardize task-specific evaluation. RAG benchmarks are typically dataset-centric (SQuAD [24], MS MARCO [19], medical RAG [29]) rather than configuration-centric. RAG-NIMBF fills this gap by treating configuration exploration as a first-class concern: given parameter ranges for all pipeline components, it generates, executes, evaluates, and tracks every combination.

3 Framework Architecture

RAG-NIMBF wraps existing RAG components without modifying their internals. The pipeline proceeds through six stages—configuration, data loading, chunking, retrieval, generation, and evaluation—each implemented as an independent module.

3.1 Configuration and Data Loading

The framework reads all parameters from a single `.env` file. Users specify comma-separated lists for each sweepable parameter (LLM models, embedding models, chunk sizes, chunking strategies, retrieval methods, rerankers, prompt templates) and the framework computes their cartesian product. Model identifiers use `provider:name` syntax (e.g., `ollama:qwen3:32b`, `openai:Qwen/Qwen3-32B-AWQ`) with automatic provider detection. Separate

base URLs can be configured per role (generation LLM, critic LLM, embedding), enabling distributed setups.

A registry-based adapter pattern maps dataset identifiers to loading functions. Each adapter converts a HuggingFace dataset into a unified `{question, ground_truth, context}` format. Built-in adapters cover SQuAD [24], RagBench, T2-RagBench, Ragas-WikiQA, and RagPerf-Wikipedia-NQ. For datasets with a shared corpus, documents are loaded once and reused across configurations.

3.2 Chunking

The framework supports three chunking strategies via LangChain text splitters [31]: *recursive* splitting (hierarchy of separators), *fixed-size* splitting (character boundaries with overlap), and *semantic* splitting (groups sentences by embedding similarity at configurable breakpoints). For semantic chunking, chunk size and overlap are ignored since boundaries are determined by embedding similarity.

3.3 Retrieval and Indexing

Chunks are embedded and stored in ChromaDB or LanceDB. Vector stores are cached using content fingerprint keys, allowing index reuse across configurations that share the same chunking and embedding setup.

Retrieval supports *similarity search* (k nearest neighbors) and *Maximal Marginal Relevance* (MMR) [3], which balances relevance with diversity through a tunable λ parameter. An optional HyDE step generates a hypothetical answer from the query and uses it as the retrieval input. An optional cross-encoder reranker re-scores retrieved documents and selects the top- n for generation.

3.4 Generation and Evaluation

The framework routes requests to Ollama or OpenAI-compatible endpoints based on the model prefix. A wrapper normalizes response formats for RAGAS compatibility. Answer post-processing strips thinking tags from reasoning models.

Each configuration is evaluated through three complementary metric layers:

LLM-based quality metrics (RAGAS). A separate critic LLM assesses faithfulness, context recall, and semantic similarity. Faithfulness is the fraction of atomic claims decomposed from the generated answer that are entailed by the retrieved context:

$$\text{Faithfulness} = \frac{|\{c_i \in \mathcal{C}_{\text{ans}} : c_i \text{ supported by } \mathcal{C}_{\text{ctx}}\}|}{|\mathcal{C}_{\text{ans}}|}. \quad (1)$$

Context recall is the fraction of ground-truth statements attributable to the context.

Information retrieval metrics. We compute $\text{Hit}@k$, $\text{Recall}@k = |\mathcal{R}_k \cap \mathcal{G}|/|\mathcal{G}|$, and $\text{nDCG}@k = \text{DCG}@k/\text{IDCG}@k$ with $\text{DCG}@k = \sum_{i=1}^k \text{rel}_i / \log_2(i + 1)$, normalised by the ideal ranking. Context relevance is cosine similarity between question and chunk embeddings. The MMR retrieval objective selects the chunk d^* maximising $\lambda \cdot \text{sim}(d_i, q) - (1 - \lambda) \cdot \max_{d_j \in S} \text{sim}(d_i, d_j)$, with S the already-selected set and $\lambda \in [0, 1]$ trading relevance for diversity (we use $\lambda = 0.5$).

NLG metrics. BERTScore-F1 [32] (RoBERTa-large) is the harmonic mean of token-level embedding precision and recall. METEOR [2], ROUGE-L [16], and BLEU [22] compare generated answers to references. Time-to-first-token (TTFT) and tokens per second (TPS) are recorded per question.

All parameters, metrics, and artifacts are logged to MLflow with a reproducibility bundle capturing the full environment state.

4 Experimental Setup

4.1 Models

We benchmark seven generation models spanning dense transformers, mixture-of-experts architectures, and local, cloud-hosted, and API deployments.

- **Qwen3-32B** — 32B dense, 4-bit AWQ via Ollama on RTX 5090.
- **Qwen3.5-397B-A17B** — MoE, 397B total / 17B active [23], vLLM on H100.
- **GPT-OSS-20B** — 20B open-source MoE (5B active) [21], Ollama on DGX Spark.
- **Qwen3.5-think** — reasoning variant with chain-of-thought traces, vLLM on H100.
- **GPT-4o-mini** — OpenAI API, upper-bound reference.
- **DeepSeek-V4-Pro** — DeepSeek API, similarity retrieval only.
- **Qwen3.6-27B** — 27B, NVFP4 quantization, vLLM on RTX 5090.

The RAGAS critic is Qwen3.5-397B-A17B in non-thinking mode on the same H100 cluster.

4.2 Embedding and Dataset

All configurations use `nomic-embed-text:latest` [20] (768-dim, MTEB-competitive [18]) via Ollama. We sample 50 questions per configuration from SQuAD [24]. The shared context corpus is loaded once and reused, so retrieval differences stem solely from chunking and embedding parameters.

Table 1. Configuration space for Phase 1 experiments. The cartesian product of the top seven rows yields 224 nominal combinations; 182 produce complete evaluation bundles analysed in Section 5.

Parameter	Values
LLM Models ($M = 7$)	Qwen3-32B, Qwen3.5-397B, GPT-OSS-20B, Qwen3.5-think, GPT-4o-mini, DeepSeek-V4-Pro, Qwen3.6-27B
Chunking Strategies ($C = 2$)	Recursive, Semantic
Chunk Sizes ($S = 2$)	500, 1000
Chunk Overlaps ($O = 2$)	100, 200
Retrieval Methods ($R = 2$)	Similarity, MMR
Prompt Templates ($T = 2$)	Concise, Detailed
Retrieval Top- k	12
Reranker / HyDE	Disabled
Nominal combinations	$M \cdot C \cdot S \cdot O \cdot R \cdot T = 224$
Analysed runs (full metric set)	182

4.3 Configuration Space

Table 1 summarizes the parameters swept in our experiments. The cartesian product of the seven sweepable parameters yields a nominal space of $M \cdot C \cdot S \cdot O \cdot R \cdot T = 224$ parameter combinations. After accounting for two structural constraints—DeepSeek-V4-Pro runs similarity retrieval only (API restriction), and semantic chunking renders chunk size/overlap inert—182 unique Phase 1 configurations produced full metric sets suitable for cross-configuration comparison. The remaining MLflow entries correspond to partial reruns, index-only stages, and failed runs excluded from analysis.

4.4 Prompt Templates

Figure 1 reproduces the two system prompts. The **Concise** template enforces a single-sentence factual answer with three WikiQA-style few-shot exemplars (two shown). The **Detailed** template removes the exemplars and instructs the model to produce a full-sentence answer that includes supporting reasoning when applicable. Both share the same human-turn format (`Context: ...Question: ...Answer:`).

4.5 Evaluation Protocol

RAGAS metrics (faithfulness, context recall, semantic similarity) are computed using Qwen3.5-397B-A17B as the critic model with a maximum of 4096 critic tokens per evaluation call. IR metrics (Hit@k, nDCG@k, Recall@k for $k \in \{1, 3, 5\}$) are computed against gold document labels provided by the SQuAD dataset. NLG metrics use RoBERTa-large for BERTScore [32] and standard implementations for METEOR [2], ROUGE-L [16], and BLEU [22]. All metrics are reported

Concise

```

Answer the question using ONLY the
provided context.
RULES:
- Give a short, direct, factual
  answer
  -- one sentence maximum.
- Do NOT explain reasoning, add
  context,
  or use markdown.
- Do NOT say 'Based on the context
  '.
- If the context does not contain
  the
  answer, say 'I cannot answer from
  the
  provided context.'
```

Examples:

Q: What circuit court is Maryland?

A: The Circuit Courts of Maryland are the state trial courts of general jurisdiction.

Q: Who wrote the music for Star Wars?

A: John Williams.

Detailed

```

Answer the question based only on
the
provided context.
Provide a clear, complete answer in
full
sentences.
Include relevant calculations or
reasoning
steps when applicable.
Do not start your answer with
phrases
like 'Based on the provided context
' or
'Based on the given context'.
Answer directly.
```

Human turn (both templates):

Context: {context}

Question: {question}

Answer:

Fig. 1. The two prompt templates, reproduced from `benchmark/prompt_templates/`. Concise imposes a one-sentence factual constraint with WikiQA-style few-shot exemplars; Detailed removes exemplars and asks for full-sentence answers with reasoning.

as means over the 50-question sample, with per-question values retained for the bootstrap and significance analyses reported in Section 5.

4.6 Reproducibility

All code, configs, prompt templates, and MLflow bundles (model/dependency versions and git hashes) are at <https://github.com/niclashart/Benchmarking-Framework>. The 182 configurations can be re-executed via the `BENCHMARK_STAGE` environment variable and the included `.env` file.

5 Results

We organize the analysis around six research questions. All metric values are means over 50 SQuAD questions per configuration; per-question bootstrap analyses and paired significance tests are reported in Section 5.7.

5.1 RQ1: How Do Generation Models Compare?

Table 2 aggregates results across all configurations for each model. For each model, we report the mean and best observed values for key metrics.

Table 2. Model comparison: mean and (best) across all configurations. Faithfulness (Faith.), nDCG@5, BERTScore F1 (BS-F1), and METEOR are reported.

Model	Faith. $\uparrow$	nDCG@5 $\uparrow$	BS-F1 $\uparrow$	METEOR $\uparrow$
Qwen3-32B	0.86 (0.92)	0.74 (0.81)	0.90 (0.94)	0.49 (0.60)
Qwen3.5-397B	0.90 (0.95)	0.76 (0.83)	0.85 (0.89)	0.47 (0.58)
GPT-OSS-20B	0.81 (0.84)	0.74 (0.80)	0.90 (0.96)	0.48 (0.60)
Qwen3.5-think	0.89 (0.95)	0.74 (0.81)	0.84 (0.87)	0.49 (0.57)
GPT-4o-mini	0.88 (0.95)	0.75 (0.80)	0.84 (0.89)	0.48 (0.57)
DeepSeek-V4-Pro	0.88 (0.94)	0.69 (0.73)	0.91 (0.92)	0.54 (0.74)
Qwen3.6-27B	0.88 (0.95)	0.75 (0.80)	0.86 (0.90)	0.49 (0.58)

Table 3. Retrieval strategy comparison: mean over all chunk sizes, overlaps, and templates. We report the five models with full coverage across both strategies; DeepSeek-V4-Pro runs similarity retrieval only (API constraint) and Qwen3.5-think is excluded from this table because its chain-of-thought traces produce a non-comparable token economy. Both are retained in the aggregate Table 2.

Model	Strategy	Faith.	nDCG@5	BS-F1	METEOR
Qwen3-32B	Similarity	0.89	0.70	0.92	0.49
	MMR	0.85	0.79	0.88	0.50
Qwen3.5-397B	Similarity	0.91	0.72	0.87	0.50
	MMR	0.88	0.80	0.83	0.45
GPT-OSS-20B	Similarity	0.82	0.70	0.92	0.49
	MMR	0.79	0.79	0.89	0.49
GPT-4o-mini	Similarity	0.89	0.70	0.86	0.52
	MMR	0.86	0.79	0.83	0.46
Qwen3.6-27B	Similarity	0.91	0.70	0.88	0.52
	MMR	0.86	0.79	0.84	0.48

Qwen3.5-397B achieves the highest mean faithfulness (0.90) and best single configuration (0.955). GPT-4o-mini, DeepSeek-V4-Pro, and Qwen3.6-27B reach comparable mean faithfulness (0.88), showing that strong factual grounding is attainable across diverse architectures. GPT-OSS-20B trails on faithfulness but reaches competitive BERTScore F1 (up to 0.96). The 17B-active MoE (Qwen3.5-397B) consistently outperforms the 32B dense Qwen3-32B, confirming the efficiency of mixture-of-experts architectures for RAG. DeepSeek-V4-Pro leads METEOR (mean 0.54, best 0.74).

5.2 RQ2: Retrieval Strategy — Similarity vs. MMR

Table 3 compares similarity search and MMR retrieval across all models.

MMR retrieval improves nDCG@5 by 9–12pp across all models by reducing redundancy. Hit@1 is unchanged (top-1 retrieval is deterministic), but Recall@k

Table 4. IR metrics for Qwen3-32B across chunk sizes and retrieval strategies (Concise template).

Size/Overlap	Retrieval	Hit@1	nDCG@5	Recall@5	Ctx. Rel.
500/100	Similarity	0.74	0.728	0.553	0.600
500/100	MMR	0.74	0.805	0.860	0.672
1000/200	Similarity	0.70	0.678	0.508	0.574
1000/200	MMR	0.70	0.772	0.820	0.645

Table 5. Prompt template comparison: mean over all chunk sizes, overlaps, and retrieval strategies. Same five-model subset as Table 3; the aggregate trends hold for the two omitted models (Qwen3.5-think, DeepSeek-V4-Pro).

Model	Template	Faith.	nDCG@5	BS-F1	METEOR
Qwen3-32B	Concise	0.85	0.74	0.89	0.58
	Detailed	0.88	0.74	0.91	0.41
Qwen3.5-397B	Concise	0.84	0.76	0.84	0.53
	Detailed	0.93	0.76	0.86	0.42
GPT-OSS-20B	Concise	0.81	0.74	0.91	0.56
	Detailed	0.81	0.74	0.91	0.34
GPT-4o-mini	Concise	0.82	0.75	0.83	0.53
	Detailed	0.93	0.75	0.85	0.42
Qwen3.6-27B	Concise	0.82	0.76	0.84	0.55
	Detailed	0.94	0.75	0.88	0.43

improves substantially because MMR surfaces a broader set of relevant documents.

MMR slightly reduces faithfulness (1–3pp) because the diversified context mixes less-relevant passages with core evidence, diluting the signal for the generation model. Table 4 shows the underlying retrieval quality: MMR improves Recall@5 from 0.55 to 0.86 (500/100 chunks)—86% of relevant context retrieved in the top-5 vs. 55% with similarity search.

5.3 RQ3: Prompt Template Effects

The choice of prompt template produces the most consistent effect across all models. Table 5 shows that the Detailed template raises faithfulness by up to 12 percentage points for capable models, while smaller models (GPT-OSS-20B) show no measurable faithfulness gain.

The faithfulness gain comes at a cost: METEOR drops by 10–22pp with Detailed for models where the template takes effect. This inversion is expected—Detailed produces thorough answers with supporting evidence, aligning with retrieved context (higher faithfulness) but diverging from the brief SQuAD references (lower METEOR). BERTScore F1 is less affected (1–3pp), confirming semantic stability. This lexical-vs-semantic divergence highlights the importance

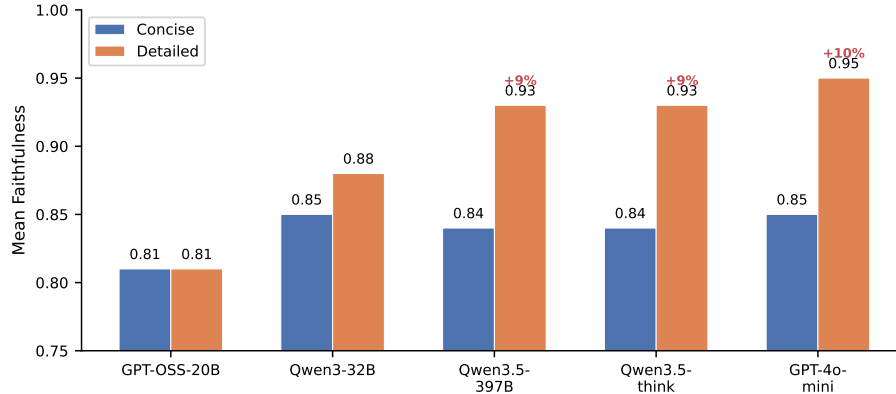


Fig. 2. Interaction between prompt template and generation model on mean faithfulness. Larger models (Qwen3.5-397B, Qwen3.5-think, GPT-4o-mini) show gains of 9–11pp with the Detailed template, while GPT-OSS-20B shows no effect.

of multi-metric evaluation: optimizing for METEOR alone would misleadingly favor concise, under-specified answers.

5.4 RQ4: Chunk Size and Overlap

We test four chunk size/overlap combinations—(500, 100), (500, 200), (1000, 100), (1000, 200)—on Qwen3-32B with recursive chunking and similarity retrieval (means over both templates). Effects are modest: faithfulness spans 0.86–0.91 and nDCG@5 spans 0.68–0.73. Smaller chunks (500) produce slightly higher nDCG@5 (+5pp) via finer granularity; larger chunks (1000) marginally raise faithfulness via broader grounding context. Semantic chunking yields identical results across all size/overlap combinations, as expected: boundaries follow embedding-similarity breakpoints, not character counts.

5.5 RQ5: Interaction Effects

The most striking finding is the interaction between prompt template and model size. Figure 2 visualizes this interaction for faithfulness.

For GPT-OSS-20B, the Detailed template has negligible effect on faithfulness (+0.0pp). For larger models (Qwen3.5-397B, Qwen3.5-think), Detailed adds 7–9pp. This suggests that larger models better follow complex instructions to ground their answers in context, while smaller models default to parametric knowledge regardless of prompt wording.

The best configuration achieves 0.955 faithfulness: Qwen3.5-397B with recursive chunking (500/100), similarity retrieval, Detailed (configuration #42). GPT-4o-mini reaches 0.951 with (1000/100) and similarity retrieval (#178). The best nDCG@5 (0.825): Qwen3.5-397B with (500/200), MMR, Detailed (#46).

Table 6. Runtime measurements from `LLM_Performance_Tests-main` (N=6, max 1024 output tokens). TPS is mean generated tokens per second.

Model	Seq. wall (s)	Par. wall (s)	Seq. TPS	Par. TPS	TTFT (s)
GPT-OSS-20B / Ollama	77.09	44.56	53.9	22.4	0.47
Qwen3-32B-AWQ / vLLM	87.13	22.23	58.9	45.4	0.30
GPT-4o-mini / API	54.93	28.76	79.4	59.1	0.45
Qwen3.5-397B-A17B / vLLM	74.97	24.05	81.7	79.1	2.10
Qwen3.5-think / vLLM	80.19	26.43	78.8	68.5	3.50
Qwen3.6-27B-NVFP4 / vLLM	38.59	24.80	127.5	70.5	1.86

5.6 RQ6: Performance–Quality Tradeoffs

Table 6 reports raw inference measurements from the separate `LLM_Performance_Tests-main` harness using six prompts, 1024 maximum output tokens, and both sequential and parallel execution. DeepSeek-V4-Pro is served via a third-party API and is excluded from the runtime table: its latency is dominated by network round-trip rather than model throughput and is not directly comparable to the other checkpoints, which run on identical hardware.

Qwen3.6-27B-NVFP4 on the RTX 5090 leads sequential throughput (127.5 TPS); Qwen3.5-397B-A17B leads parallel throughput (79.1 TPS). The local Qwen3.6 model pairs high throughput with 0.88 mean faithfulness, making it the strongest local performance–quality candidate. TTFT is deployment-sensitive: local and API models start below two seconds; the cloud thinking model reaches 3.50 s.

Figure 3 plots external throughput (x-axis) against mean faithfulness (y-axis). Qwen3.6-27B-NVFP4 anchors the high-throughput end of the frontier; Qwen3.5-397B-A17B and GPT-4o-mini anchor the high-quality end. GPT-OSS-20B sits off the frontier: moderate throughput is not redeemed by faithfulness.

5.7 Statistical Significance and Confidence Intervals

Table 7 reports 95% bootstrap CIs (2000 resamples, 50 questions per configuration) on per-question faithfulness and paired Wilcoxon signed-rank tests for the two principal within-model contrasts.

The faithfulness ranking is robust: 95% CIs are tight (± 0.01) and the top four models overlap, so sub-0.01 differences among them are not meaningful. The Detailed template produces significant faithfulness gains for five of seven models (largest on GPT-4o-mini, Qwen3.5-think, Qwen3-32B); GPT-OSS-20B shows no effect, consistent with RQ5. The MMR-vs-similarity effect is smaller, significant for only three models, with GPT-4o-mini showing a significant faithfulness *decrease* under MMR—supporting the RQ2 grounding-dilution observation.

Figure 4 visualises the headline template effect with bootstrap CIs. The 0.082 absolute gap between concise and detailed templates dwarfs the CI half-widths, confirming that the effect is large enough to be practically meaningful, not merely statistically detectable.

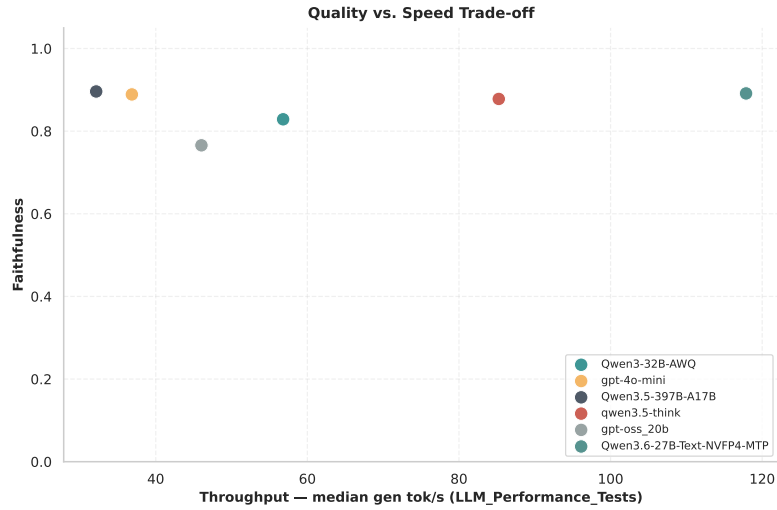


Fig. 3. Faithfulness vs. median generation throughput from the external `LLM_Performance_Tests` harness. The dashed line marks the Pareto frontier: no single model dominates on both axes, and the frontier trades quality for speed across architectures and deployments.

6 Discussion

6.1 Key Findings

Prompt design rivals model scale for faithfulness. Switching Concise→Detailed raises faithfulness by up to 12pp for capable models—comparable to upgrading Qwen3-32B to Qwen3.5-397B. For Qwen3.5-397B the template switch alone lifts mean faithfulness from 0.84 to 0.93. As prompt changes require no hardware upgrades and add no inference overhead, they should be the first optimization lever.

MMR improves coverage but requires context management. MMR improves retrieval recall and nDCG but can dilute faithfulness for smaller models: GPT-OSS-20B drops from 0.82 to 0.79 faithfulness despite nDCG@5 rising from 0.70 to 0.79, indicating the 20B model cannot reliably separate relevant evidence from peripheral passages. Practitioners should pair MMR with a larger generation model or a reranking step.

Chunk size effects are secondary. 500 vs. 1000 character chunks differ by 2–5pp on most metrics, vs. 9–12pp for retrieval strategy and up to 12pp for template choice. Retrieval and prompt optimization should precede chunk-parameter tuning.

Table 7. Per-question bootstrap 95% CIs and paired Wilcoxon p -values for template (Detailed—Concise) and retrieval (MMR—Similarity) effects. *** $p < 0.001$, ** $p < 0.01$, * $p < 0.05$, ns not significant.

Model	Faith. [95% CI]	Δ_{tmpl}	p	Δ_{retr}	p
Qwen3.5-397B	0.897 [0.890, 0.905]	+0.055	0.0035 **	+0.049	0.006 **
Qwen3.6-27B	0.891 [0.882, 0.900]	+0.092	<0.001 ***	+0.012	0.27 ns
GPT-4o-mini	0.889 [0.880, 0.898]	+0.128	<0.001 ***	−0.042	0.001 ***
DeepSeek-V4-Pro	0.881 [0.861, 0.902]	+0.074	0.031 *	—	—
Qwen3.5-think	0.878 [0.867, 0.888]	+0.126	<0.001 ***	−0.006	0.63 ns
Qwen3-32B	0.843 [0.833, 0.853]	+0.121	<0.001 ***	+0.053	0.014 *
GPT-OSS-20B	0.786 [0.774, 0.799]	−0.040	0.069 ns	+0.002	0.98 ns

6.2 Metric Divergence

Lexical metrics (METEOR, ROUGE-L, BLEU) diverge from semantic metrics (BERTScore, faithfulness) across templates: Detailed lowers METEOR by 10–22pp while raising faithfulness by up to 12pp, because SQuAD references are short noun phrases while Detailed answers include supporting context. BERTScore F1 varies by only 1–3pp, confirming semantic stability. Metric selection should follow the application’s verbosity tolerance.

6.3 Threats to Validity

Internal. LLM-based metrics depend on the Qwen3.5-397B critic, which may favour same-family outputs; we report IR and NLG metrics as complementary signals.

External. Results are obtained on SQuAD, a single-domain extractive QA dataset; abstractive or domain-specific tasks may differ. The framework supports additional datasets (RagPerf-Wikipedia-NQ, T2-RagBench, FinQA).

Statistical. Bootstrap 95% CIs (Table 7) are tight (± 0.01); paired Wilcoxon tests confirm headline template and retrieval effects at $p < 0.05$ for five of seven models. Sub-0.01 differences between top-tier models fall within CI overlap.

Reproducibility. We use default server temperatures without fixed seeds; the reproducibility bundle captures model/dependency versions and git hash, but exact numerical replication may vary slightly due to non-deterministic GPU operations.

6.4 Limitations

The study excludes reranking, HyDE query expansion, and embedding-model comparisons. All quality assessments are automated; no human evaluation is included.

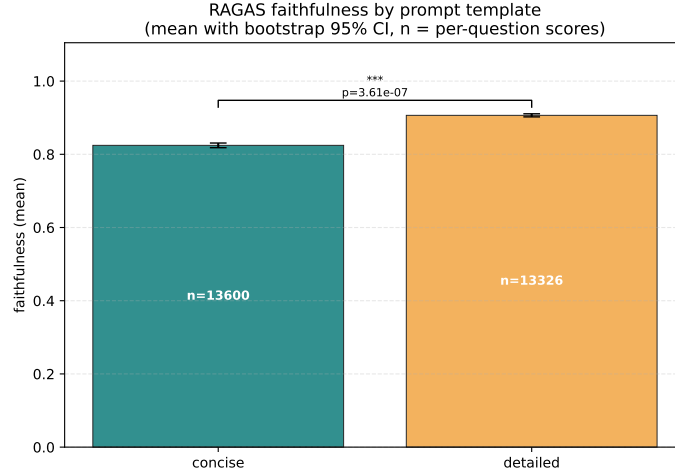


Fig. 4. Per-question bootstrap 95% CIs (2000 resamples) on faithfulness, pooled across all configurations by prompt template. The Detailed template raises faithfulness significantly (paired Wilcoxon $p < 0.001$, $***$); CIs do not overlap.

7 Conclusion

We presented RAG-NIMBF, a non-invasive, modular benchmarking framework for systematic RAG configuration analysis. Across 182 SQuAD configurations, prompt specificity was the strongest lever for faithfulness, MMR improved retrieval coverage while sometimes reducing grounding, and chunk-size effects were secondary to prompt and retrieval choices. Multi-metric evaluation is necessary: lexical metrics favor concise SQuAD-style answers, whereas faithfulness and BERTScore reward context-grounded semantic content.

Runtime measurements from the independent LLM performance harness show that local deployment can be competitive for batch evaluation, especially with Qwen3.6-27B-NVFP4 on consumer hardware. Future work will extend evaluation to additional datasets, reranking, HyDE query expansion, embedding-model comparisons, and larger samples.

Declarations

Data Availability. All experimental artefacts (source code, configuration files, prompt templates, raw per-question results, MLflow tracking bundles) are publicly available at <https://github.com/niclashart/Benchmarking-Framework>. SQuAD [24] is available via HuggingFace Datasets.

Competing Interests. The authors declare no competing interests.

Funding. No external funding. Computational resources were provided by Ansbach University of Applied Sciences.

Ethics Declaration. This study uses only the publicly available SQuAD

dataset and does not involve human participants, personal data, or sensitive information.

Author Contributions. **N. Hart:** Conceptualization, Methodology, Software, Investigation, Formal Analysis, Visualization, Writing – Original Draft. **N. Weeger:** Data Curation, Validation, Investigation, Writing – Review & Editing. Both authors read and approved the final manuscript.

Generative AI Use. Claude Code (Anthropic) and GitHub Copilot assisted with software development and manuscript drafting. All scientific content, experimental design, and analysis are the authors’, who reviewed and verified all AI-generated text and take responsibility for the final manuscript.

References

1. Asai, A., Wu, Z., Wang, Y., Sil, A., Hajishirzi, H.: Self-RAG: Learning to retrieve, generate, and critique through self-reflection. In: International Conference on Learning Representations (2024)
2. Banerjee, S., Lavie, A.: METEOR: An automatic metric for MT evaluation with improved correlation with human judgments. Proceedings of the ACL Workshop on Intrinsic and Extrinsic Evaluation Measures for Machine Translation and/or Summarization pp. 65–72 (2005)
3. Carbonell, J., Goldstein, J.: The use of MMR, diversity-based reranking for re-ordering documents and producing summaries. In: Proceedings of the 21st Annual International ACM SIGIR Conference on Research and Development in Information Retrieval. pp. 335–336 (1998)
4. Chen, J., Lin, H., Han, X., Sun, L.: Benchmarking large language models on retrieval-augmented generation. arXiv preprint arXiv:2404.14405 (2024)
5. Confident AI: DeepEval: The LLM evaluation framework. <https://github.com/confident-ai/deepeval> (2024), software, accessed 2026-06
6. Edge, D., Trinh, H., , Cheng, N., Bradley, J., Chao, A., et al.: From local to global: A graph RAG approach to query-focused summarization. arXiv preprint arXiv:2404.16130 (2024)
7. Es, S., James, J., Espinosa-Anke, L., Schockaert, S.: RAGAS: Automated evaluation of retrieval augmented generation. arXiv preprint arXiv:2309.15217 (2023)
8. Gao, Y., Xiong, Y., Dixit, A., Gao, X., Liu, K., et al.: Retrieval-augmented generation for large language models: A survey. In: arXiv preprint arXiv:2312.10997 (2024)
9. Guu, K., Lee, K., Tung, Z., Pasupat, P., Chang, M.W.: REALM: Retrieval-augmented language model pre-training. In: International Conference on Machine Learning. pp. 3929–3938 (2020)
10. Jiang, Y., et al.: Active retrieval augmented generation. In: Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing (2024)
11. Karpukhin, V., Öguz, B., Min, S., Lewis, P., Wu, L., Edunov, S., Chen, D., Yih, W.t.: Dense passage retrieval for open-domain question answering. In: Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing. pp. 6769–6781 (2020)
12. LangChain Inc.: LangSmith: Tracing and evaluation for LLM applications. <https://docs.smith.langchain.com> (2024), software, accessed 2026-06
13. Langfuse GmbH: Langfuse: Open-source LLM engineering platform. <https://langfuse.com> (2024), software, accessed 2026-06
14. Lewis, P., Perez, E., Piktus, A., Petroni, F., Karpukhin, V., Goyal, N., Küttler, H., Lewis, M., Yih, W.t., Rocktäschel, T., Riedel, S., Kiela, D.: Retrieval-augmented

- generation for knowledge-intensive NLP tasks. In: *Advances in Neural Information Processing Systems*. vol. 33, pp. 9459–9474 (2020)
15. Li, W., et al.: Chain-of-note: Reducing hallucination in retrieval augmented generation. *arXiv preprint arXiv:2311.09243* (2024)
 16. Lin, C.Y.: ROUGE: A package for automatic evaluation of summaries. In: *Text Summarization Branches Out*. pp. 74–81 (2004)
 17. Liu, N.F., Lin, K., Hewitt, J., Paranjape, A., Belezak, M., Petroni, F., Zettlemoyer, L.: Lost in the middle: How language models use long contexts. *Transactions of the Association for Computational Linguistics* **12**, 157–173 (2024)
 18. Muennighoff, N., Tazi, N., Magne, L., Reimers, N.: MTEB: Massive text embedding benchmark. *arXiv preprint arXiv:2210.07316* (2022)
 19. Nguyen, T., Rosenberg, M., Song, X., Gao, J., Tiwary, S., Majumder, R., Deng, L.: MS MARCO: A human generated machine reading comprehension dataset. In: *CEUR Workshop Proceedings*. vol. 1773 (2016)
 20. Nomic AI: nomic-embed-text: A high-performance open embedding model. Technical Report (2024), <https://huggingface.co/nomic-ai/nomic-embed-text-v1.5>
 21. Open Source Community: GPT-OSS-20b: An open-source large language model. HuggingFace Model Repository (2024), <https://huggingface.co/GPT-OSS>
 22. Papineni, K., Roukos, S., Ward, T., Zhu, W.J.: BLEU: A method for automatic evaluation of machine translation. *Proceedings of the 40th Annual Meeting of the Association for Computational Linguistics* pp. 311–318 (2002)
 23. Qwen Team: QwQ: Reflect, reason, respond. Technical Report, Alibaba Cloud (2025), <https://qwenlm.github.io/blog/qwq-32b/>
 24. Rajpurkar, P., Zhang, J., Lopyrev, K., Liang, P.: SQuAD: 100,000+ questions for machine comprehension of text. In: *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing*. pp. 2383–2392 (2016)
 25. Reimers, N., Gurevych, I.: Sentence-BERT: Sentence embeddings using siamese BERT-networks. In: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing*. pp. 3982–3992 (2019)
 26. Ru, D., Qiu, L., Huang, X., Feng, J., Wang, Q., et al.: RAGChecker: A fine-grained framework for diagnosing retrieval-augmented generation. *arXiv preprint arXiv:2408.08067* (2024)
 27. Saad-Falcon, J., Khramtsova, O., Lin, M.O., Cristian, A.R., Khashabi, D.: ARES: An automated evaluation framework for retrieval-augmented generation systems. *arXiv preprint arXiv:2311.09476* (2024)
 28. Shuster, K., Poff, S., Chen, M., Kiela, D., Weston, J.: Retrieval augmentation reduces hallucination in conversation. In: *Findings of the Association for Computational Linguistics: EMNLP 2021*. pp. 3784–3803 (2021)
 29. Song, S., et al.: Benchmarking retrieval-augmented generation for medicine. *arXiv preprint arXiv:2402.13178* (2024)
 30. TruEra: TruLens: Evaluation and tracking for LLM experiments. <https://github.com/truera/trulens> (2023), software, accessed 2026-06
 31. Wolf, T., Debut, L., Sanh, V., Chaumond, J., Delangue, C., Moi, A., Cistac, P., Rault, T., Louf, R., Funtowicz, M., Davison, J., Shleifer, S., von Platen, P., Cao, C., Jernite, Y., Plu, J., Xu, C., Le Scao, T., Gugger, S., Drame, M., Lhoest, Q., Rush, A.M.: Transformers: State-of-the-art natural language processing. In: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing: System Demonstrations*. pp. 38–45 (2020)
 32. Zhang, T., Kishore, V., Wu, F., Weinberger, K.Q., Artzi, Y.: BERTScore: Evaluating text generation with BERT. In: *International Conference on Learning Representations* (2020)
 33. Zhao, P., Wang, H., Lv, J., Qin, C., Zhang, J.: Retrieval-augmented generation for AI-generated content: A survey. *arXiv preprint arXiv:2402.19473* (2024)